MongoDB Reliability and Recovery

A resident sees a confirmation.
The next page cannot find the request.

Day 1: acknowledgment, reads, and failure behavior
Day 2: recovering correct documents and their rules

Primary, secondaries, and the oplog



The oplog carries changes. Copies can lag, and election and reconnection take time.

Retaining a write and applying it

Moment	A collection	B collection	C collection
Before submission	Absent	Absent	Absent
A applies the insert	Present	Absent	Absent
B retains the oplog entry	Present	Not yet applied	Absent
B applies the insert	Present	Present	Absent

A and B can acknowledge their durable oplog entries in the third data row.
B may still need to apply the insert before a collection read can see it.

MongoDB 8.0+ with default majority journaling. Supplied state trace.

Write concern changes the acknowledgment

Setting in this case	Acknowledgment required	A limitation
w: 1	The primary	The write can lack a second retained copy
w: "majority"	Two voting, data-bearing members	The required members must be reachable
w: 3	All three data-bearing members	C is isolated, so this cannot complete now

A timeout can stop the acknowledgment wait without undoing the write. Journaling and the deployment configuration also affect durability.

A write with a stable submission identifier

```
const reliabilityPracticeName =
  "cst4714_reliability_" + ObjectId().toHexString().slice(-8);
db = db.getSiblingDB(reliabilityPracticeName);
db.tickets.insertOne(
  { _id: "submission-1099", ticket_id: 1099,
    status: "new", subject: "Write concern test" },
  { writeConcern: { w: "majority", wtimeout: 5000 } }
);
```

Mongosh illustration in a new practice database.
The timeout is 5,000 milliseconds for the requested write concern.
A normal acknowledgment does not demonstrate failover.

An uncertain write needs a read-back strategy

```
db.tickets.findOne(
  { _id: "submission-1099" },
  { _id: 1, ticket_id: 1, status: 1, subject: 1 }
);
```

Check the saved identifier and the intended content.
Reusing this _id cannot create a second document with that same _id.

A duplicate-key error alone does not prove the existing request is yours.
A missing read from a lagging copy does not prove the write failed.

Read routing and visible state are separate

Control	Example	Meaning
Read preference	primary	Select the primary for this read
Read preference	secondaryPreferred	Prefer an eligible secondary
Read concern	local	May include data not majority committed
Read concern	majority	Return majority-committed data

A lagging member can hold older majority-committed data.
Majority read concern alone does not make every copy current.

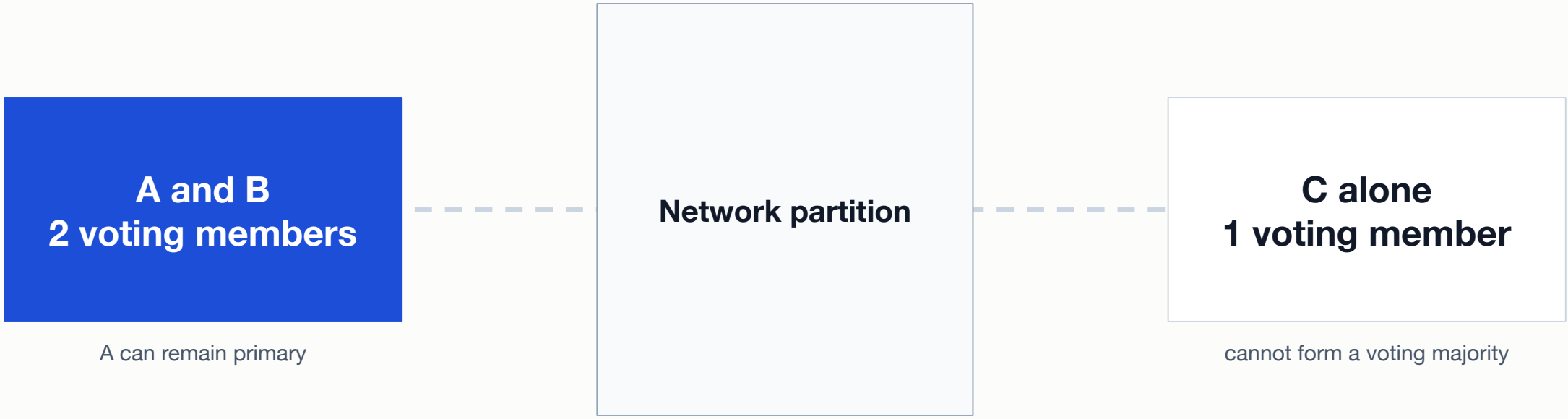
A causal session connects the confirmation read

Part of this workflow	Concrete choice
Write concern	majority
Read preference	primary
Read concern	majority
Operation relationship	Write and following read use the same causal session

The later read carries a dependency on the earlier write.
A suitable secondary read may wait to catch up instead of returning old state.

Timeouts and later changes by other requests remain possible.

A and B remain connected while C is isolated



C cannot acknowledge a new write that it has not received from A and B.

The partition creates a specific consistency conflict

A write changes ticket status to resolved on the A/B side.
That write completes before a new read begins on the C side.

C's possible behavior	Consequence
Return the old status, open	The read can violate the single-current-copy promise
Wait or reject until coordination is possible	That operation is unavailable in the CAP sense

CAP consistency means linearizable behavior, not the C in ACID.
CAP availability is not an uptime percentage.

A replacement primary needs an election

Failure state	What the members can do
A is lost; B and C can communicate	Eligible members can elect a primary with two votes
A is lost; B and C cannot communicate	Neither remaining member alone has a voting majority

Failure detection, election, and driver reconnection take time.
Some operations may fail or wait while there is no writable primary.

Supported driver retries help with particular operations.
They do not make every application action safe to repeat.

An authorized deletion can reach every replica

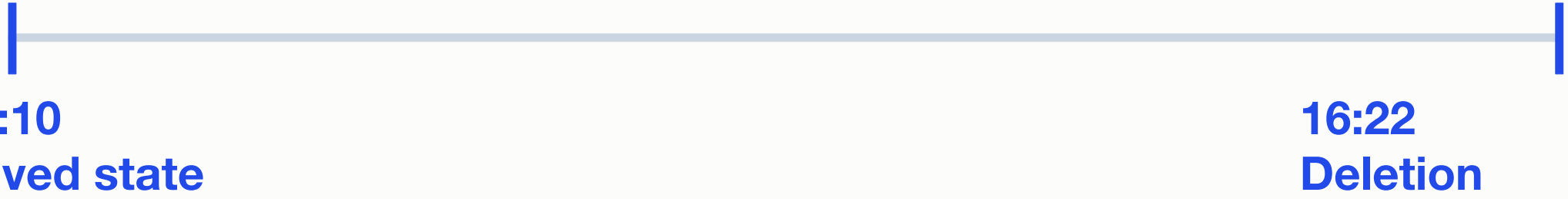
State	A	B	C	Earlier export
Before deletion	5 tickets	5 tickets	5 tickets	5 saved tickets
After deletion replicates	0 tickets	0 tickets	0 tickets	Still 5 saved tickets

Replication correctly carries out the destructive operation.
Recovering older data requires a separate retained artifact.

An oplog has a bounded history. It is not an indefinite recovery archive.

Recovery point objective (RPO)

A complete artifact represents 16:10. Deletion happens at 16:22.
No later export or replayable event log exists in this case.




```
Recovery-point gap = 16:22 − 16:10 = 12 minutes
```

A five-minute data-loss objective is not met by this artifact.

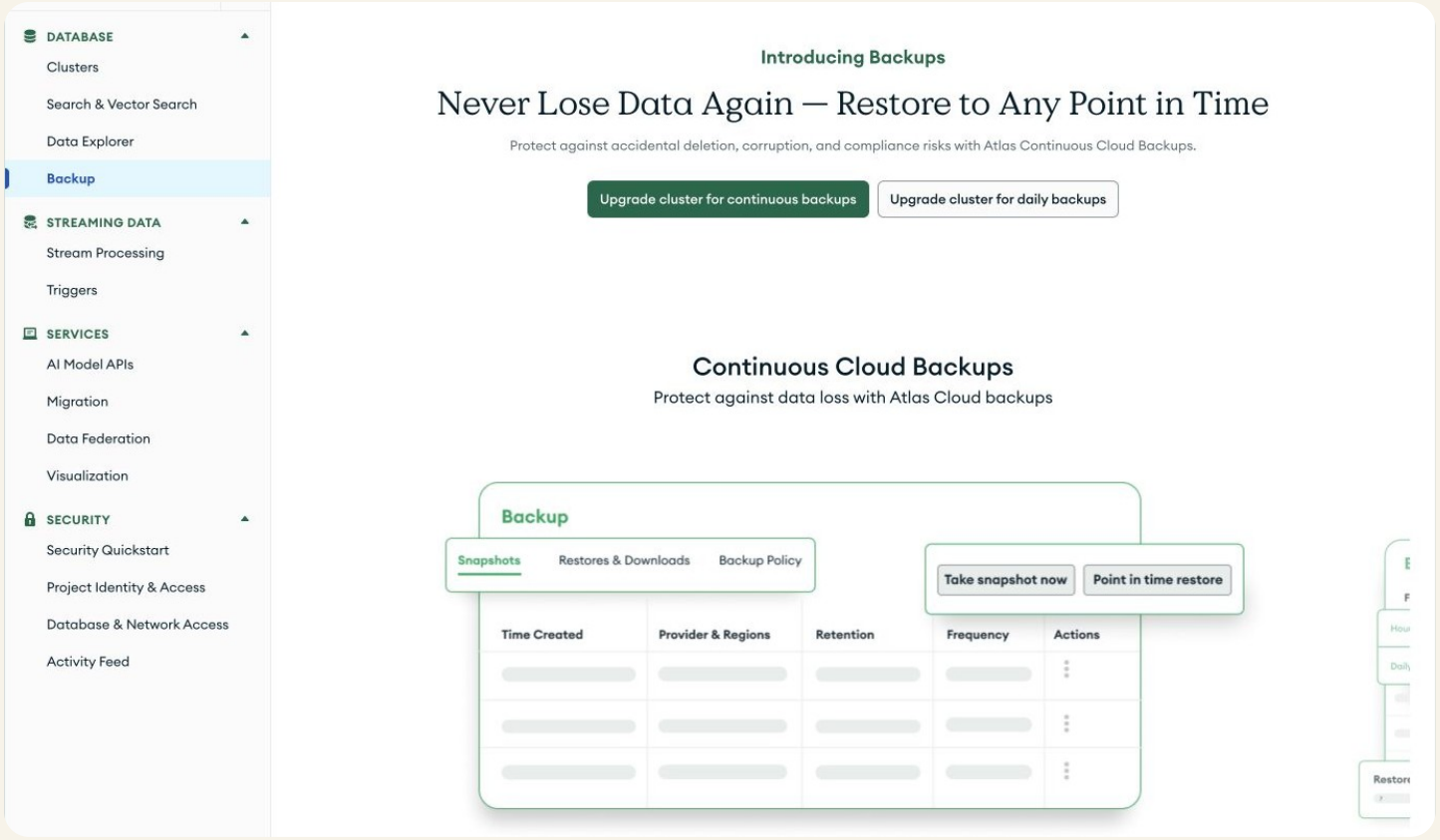
Recovery time objective (RTO)

Work after the incident	Duration
Detect the deletion	2 minutes
Retrieve and authorize the artifact	1 minute
Restore and pass application checks	4 minutes

```
Observed recovery time = 2 + 1 + 4 = 7 minutes
```

This drill meets a ten-minute service-restoration objective.
It still misses the five-minute data-loss objective from the previous slide.

The Atlas Free backup screen offers an upgrade



No native Atlas backup on the Free cluster.

Use a separate logical recovery artifact and a tested restore target.

Interface captured August 25, 2026. Documentation checked September 14.
These are upgrade choices. No completed backup appears here.

Lab 1: confirmation and recovery decisions

Choose the read/write policy for the supplied A/B/C case.
Analyze the lab's recovery timeline using its own times.
Submit one Brightspace text response, individually.

Use the supplied member states and incident times.
Keep your cloud deployment running normally.

Project discussion uses the single final-project page in the week guide.
Apply the reasoning to one important operation in your existing project notes.

Day 2: Document Recovery and Collection Rules

Five restored tickets can have the right IDs and still contain the wrong information.

The notebook traces the saved values, repairs one document, and tests the rules that the data file omitted.

The notebook can run locally or use Atlas

Path	What runs	A limit to remember
Local: USE_ATLAS = False	Data recovery, repair, unique-index test	Schema outcomes are a supplied trace
Atlas: USE_ATLAS = True	The same work plus server schema tests	Network access and permissions are required

For Atlas, add the notebook runtime IPv4 as a temporary /32 entry.
Pause until it is active, then enter the URI in the hidden prompt.
Keep TLS verification enabled. Your laptop may have a different IP.

Five fresh tickets in the recovery notebook

Ticket	Status	Subject	Opened (UTC)
1001	open	Streetlight dark near bus stop	Feb 1, 23:10
1002	in_progress	Missed recycling pickup	Feb 2, 15:45
1003	resolved	Low water pressure	Feb 3, 12:05
1004	new	Broken bench slat	Feb 4, 17:20
1005	resolved	Overflowing corner bin	Feb 5, 14:00

Active IDs: 1001, 1002, 1004. All five documents should survive recovery.
This fixture is separate from Week 11's four-ticket example.

Canonical Extended JSON carries type markers

```
{
  "ticket_id": { "$numberInt": "1001" },
  "opened_at": {
    "$date": { "$numberLong": "1769987400000" }
  }
}
```

Selected fields from ticket 1001, not the complete document.
The date number is milliseconds since the Unix epoch.
These markers let a BSON-aware parser reconstruct the stored types.

The parser determines the reconstructed values

```
date_text = (
    '{"opened_at": {"$date": '
    '{"$numberLong": "1769987400000"}}}'
)
plain = json.loads(date_text)
typed = json_util.loads(date_text, json_options=JSON_OPTIONS)
print(type(plain["opened_at"]).__name__)
print(type(typed["opened_at"]).__name__)
```

Outputs: dict, then datetime

The notebook imports json/json_util and defines JSON_OPTIONS before this work.

The source, artifact, and target remain distinct

Source collection

5 tickets
Indexes and validator

Document artifact

5 saved documents
Canonical Extended JSON

Separate target

5 recovered tickets
Initially only the _id_ index

The notebook keeps the metadata manifest separately.
It checks the artifact hash and parses the data before resetting its target.
An unchanged hash identifies bytes, not completeness or authenticity.

A wrong subject survives three weak checks

Check after the incorrect migration	Result
Five documents	True
Expected ticket IDs	True
All opening values are datetime	True
All saved document values and types match	False

Saved 1001: Streetlight dark near bus stop
Restored 1001: Text replaced by an incorrect migration

The wrong subject is still a string, so the schema alone cannot detect this error.

Restoring one document with replace_one

```
saved_by_id = {
    doc["ticket_id"]: doc for doc in restored_documents
}
recovery_document = saved_by_id[DAMAGED_TICKET]
repair = restore_tickets.replace_one(
    {"ticket_id": DAMAGED_TICKET}, recovery_document
)
print(repair.matched_count, repair.modified_count)
```

First repair: 1 matched, 1 modified. Repeat: 1 matched, 0 modified.
Five tickets remain, with the saved values restored.

Use the verified file parse. The same saved _id identifies the existing document.

Documents do not contain collection rules

Collection feature	Source	Document-only restore
Automatic _id_ index	Present	Created automatically
Unique ticket_id index	Present	Missing
Status/date compound index	Present	Missing
Server schema validator (Atlas path)	Installed	Missing

The notebook supplies definitions separately from the document JSON.
An index name alone would not describe its keys, order, or uniqueness.

Reconstruction of indexes and validation rules

```
restore_tickets.create_index(
    "ticket_id", unique=True, name="unique_ticket_id"
)
restore_tickets.create_index(
    [("status", 1), ("opened_at", -1)], name="status_by_date"
)
if USE_ATLAS:
    restore_database.command(
        "collMod", "tickets", validator=ticket_validator,
        validationLevel="strict", validationAction="error",
    )
```

ticket_validator is the notebook's defined schema. Local mode skips collMod.
Installing a rule does not prove the restored subject is correct.

The restored rules need allowed and rejected tests

Test	Expected behavior
Valid new ticket 1099	Accept
Second document with ticket_id 1001	Reject duplicate key: 11000
1100: status almost_done	Reject validation: 121
1101: date stored as text	Reject validation: 121
1102: missing opened_at	Reject validation: 121

Both paths test the unique index. Atlas also executes the schema tests.
Local schema output is a supplied trace, not server enforcement.
The notebook removes temporary test documents and keeps the five tickets.

The recovery artifacts have different scopes

Artifact	Documents / rows	Rules in its scope	Hosted project settings
This Extended JSON file	Selected collection	Rebuilt separately	No
MongoDB logical dump	Selected namespaces	Metadata and indexes	Not the whole project
Week 8 PostgreSQL archive	Selected database content	Schema and constraints	Not the whole project

The classroom data stays unchanged during export.
A busy application also needs a way to capture related data consistently and a plan for switching the application to its restored database.

Lab 2: one repair and one recovery recommendation

In Notebook 05, choose ticket 1001 or 1004 and complete the repair.
Explain the missed error, a separately restored rule, and the archive difference.
Submit the completed notebook in Brightspace, individually.

```
source_database.drop_collection("tickets")
restore_database.drop_collection("tickets")
EXPORT_FILE.unlink(missing_ok=True)
client.close()
```

Remove the temporary Atlas IP rule separately and clear its printed output.
No separate report, data-file attachment, or shared-cluster deletion.

Describing the recovery work in an interview

I restored a small collection into a separate target and found that the right count did not prove the values were correct. I recovered the damaged document from the saved artifact, checked its typed values, and tested the collection rules separately. I can explain which failure this recovery method addresses and which parts of a hosted deployment it leaves out.

Use the account that matches your own work and execution path.
Interpreting a supplied schema trace is different from testing a server rule.
Next week: capacity, sharding, and safe Python data integration.